Azure End-to-End Correlation Reference Pattern

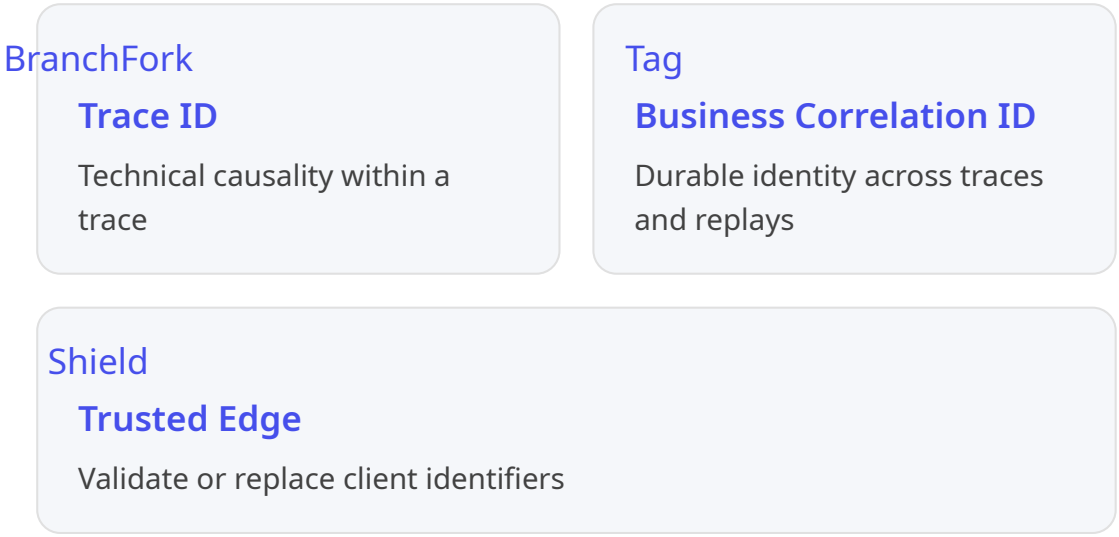
Executive Summary

Research date: October 31, 2026.

Use **two complementary correlation layers**:

1. **Technical causality — W3C Trace Context**
   - Propagate `traceparent` and, when present, `tracestate`.
   - Keep one `trace-id` across a distributed operation; create a new `span-id` at each participating hop.
   - Application Insights maps W3C `trace-id` to `OperationId`, a request/dependency span to `Id`, and its parent span to `ParentId`. Request and dependency telemetry are the actual distributed-tracing span records in Application Insights. [1] [2] [3]
2. **Durable business/user-invocation identity**
   - Carry a separately generated, immutable `x-business-correlation-id`.
   - Persist it on every span/log as `business.correlation_id` and in every message envelope/application-property set.
   - Use it to correlate delayed, retried, dead-lettered, replayed, fan-out, batch, and multi-trace processing. Service Bus explicitly provides application-defined metadata and a `CorrelationId`; OpenTelemetry also defines a messaging conversation/correlation identifier distinct from trace context. [4] [5]

Primary recommendation: Azure Monitor Application Insights + the **Microsoft OpenTelemetry Distro** for new .NET, Node.js, and Python services, and the **Azure Monitor OpenTelemetry Distro/agent for Java**. Keep the Application Insights JavaScript SDK for browser telemetry. Microsoft's current guidance explicitly directs new .NET, Node.js, and Python applications to the Microsoft distro and warns against initializing duplicate distros/providers. [6] [7]



1. Recommended GA Architecture

The recommended pattern is illustrated below. Solid HTTP hops participate in W3C tracing; asynchronous transports carry message creation context plus the durable business identifier. Application Insights uses service/resource metadata for topology and `OperationId` / `Id` / `ParentId` for transaction reconstruction. [6] [8] [5]

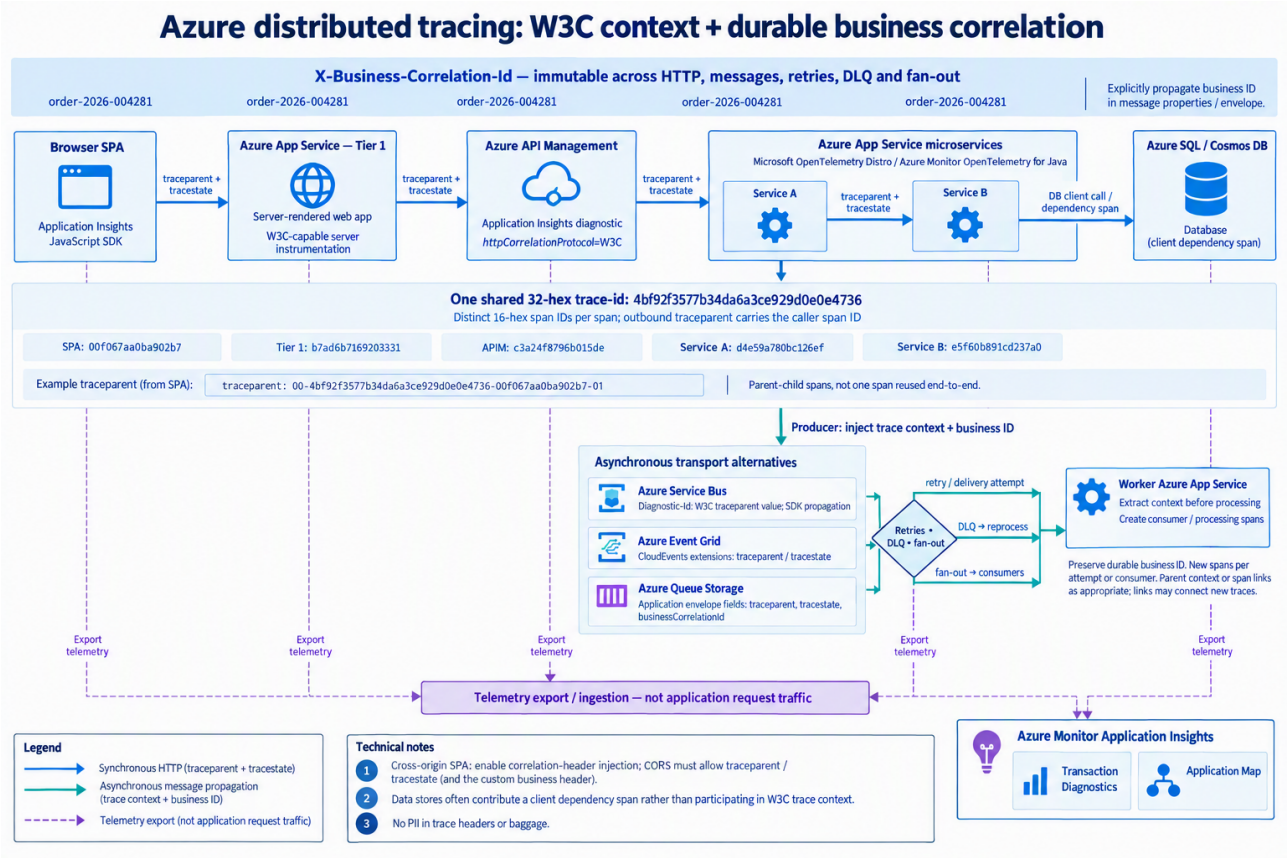


Figure 1: Recommended GA correlation pattern: W3C trace context preserves technical causality, while an immutable business invocation ID survives long-running, delayed, retried, and fan-out processing.

Concise sequence

```
Browser interaction
  ├─ SPA:
  │    App Insights JS SDK creates page/AJAX context
  │    -> traceparent + tracestate + x-business-correlation-id
  │
  └─ Server-rendered App Service:
       inbound server span
       -> outbound HTTP client span

Azure API Management
  -> APIM request span
  -> APIM backend dependency span
  -> W3C trace-id preserved; span-id advances

Downstream App Service
  -> server request span
  -> dependency span(s)
       -> Azure SQL / Cosmos DB / HTTP / Storage

Asynchronous branch
```

```
-> producer/send span
-> Service Bus / Event Grid / Queue Storage
-> consumer/process span or linked trace
-> downstream dependencies
```

Component responsibility matrix

Component	Creates	Propagates/ preserves	Consumes/ records	Required configuration
Browser SPA	Page-view/ AJAX context and client dependency	`traceparent`, `tracestate`; business ID	Page views and browser dependencies	Application Insights JS SDK; explicit W3C mode; enable and allow-list CORS correlation
Server-rendered Tier 1 App Service	Server request span; outbound client spans	W3C HTTP context; business ID	Requests, dependencies, logs	App Service autoinstrumentation or Microsoft/ Azure Monitor OTel Distro
APIM	Gateway request and backend dependency telemetry	W3C context when diagnostic protocol is `W3C`	Request, dependency, exception and configured trace telemetry	AI logger + diagnostic; `httpCorrelationProtocol: W3C`; deliberate sampling

Component	Creates	Propagates/ preserves	Consumes/ records	Required configuration
Downstream App Services	Server spans and dependency spans	Incoming/ outgoing W3C context	Requests, dependencies, exceptions, logs	One instrumentation path; stable service/ resource attributes
Service Bus producer/ consumer	Send, receive/ process and settlement operations where supported	Message trace context using `Diagnostic-Id`; durable ID in application metadata	Messaging dependencies/ process telemetry	Supported Azure SDK instrumentation; verify language/ runtime behavior
Event Grid	Publish/ delivery context	CloudEvent `traceparent` / `tracestate` extension when tracing enabled	Producer/ delivery/ consumer telemetry where instrumented	Supported SDK; mapping required for Event Grid → Service Bus/ Event Hubs
Queue Storage	Azure SDK API dependency span	**Not a per-message trace-context carrier by itself**	Queue API dependency	Put trace context and business ID inside the message envelope

Component	Creates	Propagates/ preserves	Consumes/ records	Required configuration
Data store	Usually no new server-side W3C participant	Usually none	Client dependency span and datastore-specific attributes	Supported client instrumentation; manual spans only for unsupported libraries
Application Insights	—	—	Reconstructs transactions and topology	Workspace-based resources; consistent service identity; query access

App Service autoinstrumentation currently covers supported Windows/Linux code deployments for .NET, Java and Node.js, with Python support on Linux code. Windows-container autoinstrumentation is Public Preview, and multi-container/ sidecar scenarios require code-based instrumentation. [9]

---

2. Browser SPA Versus Server-Rendered Tier 1

Browser SPA

The Application Insights JavaScript SDK:

- Automatically collects XHR and Fetch dependencies unless disabled.
- Supports `W3C`, legacy AI, or `AI_AND_W3C`; the current default is `AI_AND_W3C`.
- Automatically sends W3C context for same-origin requests.
- Requires `enableCorsCorrelation: true` for cross-origin requests.

- Provides `correlationHeaderDomains`, excluded domains, and exclusion patterns to constrain where headers are sent. [1]

Recommended SPA configuration:

```
{
  distributedTracingMode: DistributedTracingModes.W3C,
  enableCorsCorrelation: true,
  correlationHeaderDomains: [
    "api.contoso.example"
  ]
}
```

At APIM, allow the browser preflight to use `traceparent`, `tracestate`, and `x-business-correlation-id`. Use an explicit allow-list rather than sending correlation headers to arbitrary domains. The W3C specification warns that restricted cross-origin headers can cause calls to fail and that trace headers can expose cross-request relationships; neither `traceparent` nor `tracestate` may contain PII or sensitive data. [2]

A user interaction is therefore associated as:

```
AppPageViews.Id
    -> browser AppDependencies.ParentId
    -> browser dependency Id
    -> APIM AppRequests.ParentId
```

All should share the same `OperationId` when the browser-to-APIM trace context is successfully accepted. The relevant workspace tables expose `Id`, `OperationId`, `ParentId`, `Properties`, user and session fields. [10] [11] [12]

Server-rendered App Service

A server-rendered request starts or continues a **server request span** in the Tier 1 process. Instrumented `HttpClient`, Node HTTP, Java HTTP clients, Python Requests/HTTPX, and supported Azure SDK calls produce child dependency spans and inject W3C context. Microsoft's distro includes ASP.NET Core request, `HttpClient`, `SqlClient`, and Azure SDK instrumentation; other runtimes have documented bundled libraries. [7]

Unlike the SPA, server-rendered outbound calls are not governed by browser CORS. They are still subject to proxy/header filtering and trust-boundary rules.

---

3. Azure API Management

GA behavior

After Application Insights integration and API diagnostics are enabled, APIM emits:

- A **request** for the frontend request/response.
- A **dependency** for backend forwarding and other supported external policy operations.
- Exceptions for failed requests.
- Trace telemetry from the APIM `trace` policy when its severity meets the configured diagnostic verbosity. [13]

Supported policy dependency tracking includes `forward-request`, `send-request`, `send-service-bus-message`, `sql-data-source`, `cosmosdb-request-handler`, managed-identity authentication and several validation/data-source policies. These are recorded as correlated child operations, including retries. [13]

Configure:

```
properties: {
  loggerId: aiLogger.id
  httpCorrelationProtocol: 'W3C'
  alwaysLog: 'allErrors'
  sampling: {
    samplingType: 'fixed'
    percentage: 100   // verification phase
  }
}
```

The diagnostic contract formally supports `Legacy`, `None`, and `W3C`; `W3C` instructs APIM diagnostics to use Trace Context headers. [14] [15]

Business ID handling at APIM

Recommendation—not a built-in W3C requirement:

- Accept `x-business-correlation-id` only from authenticated, trusted callers and only after format/length validation.
- Otherwise replace it at the trusted edge with a server-generated opaque identifier.
- Preserve it to the backend with `set-header`.
- Emit it through a `trace` policy metadata property for APIM-side searchability.

APIM's `set-header` policy can add, replace, skip, append, or delete request/response headers; a `trace` policy can create Application Insights trace telemetry with custom metadata. [16] [17]

Do **not** synthesize `traceparent` manually from the business ID. Let APIM's W3C diagnostic implementation and the instrumentation SDK generate valid span IDs and sampling flags.

APIM failure points

1. `httpCorrelationProtocol` is `None` or remains on `Legacy`.
2. A policy explicitly deletes or overwrites `traceparent` / `tracestate`.
3. Browser CORS does not allow the trace or business headers.
4. Diagnostics are enabled at the wrong scope; API-level settings can override all-API settings.
5. APIM sampling omits its request/dependency telemetry even though downstream telemetry exists.
6. Request/response header or body logging imposes unnecessary load or captures sensitive data.
7. Self-hosted gateway telemetry can be lost because APIM does not guarantee every buffered telemetry item is flushed. [13]

Microsoft reports a substantial throughput impact when logging every APIM event at high request rates and recommends sampling and avoiding unnecessary headers/bodies. For an end-to-end proof, however, temporarily use 100% sampling on a controlled test path. [13]

---

4. Downstream App Service Instrumentation

Recommended direction

Situation	Recommendation
New .NET, Node.js or Python service	Microsoft OpenTelemetry Distro
Existing Azure Monitor OTel Distro	Retain; migrate deliberately rather than duplicate
Java	Azure Monitor OpenTelemetry Distro/ agent or supported native integration
Existing Application Insights .NET 2.x	Upgrade to 3.x or migrate cleanly
Simple supported App Service deployment	Autoinstrumentation is acceptable
Need custom spans, enrichment, filtering or dual export	Code-based OTel distro

Microsoft explicitly says not to initialize two distros or duplicate providers for the same signal. Mixed classic and OTel instrumentation can create duplicate spans, inconsistent sampling, and conflicting context. [6] [18]

Service identity

Set stable:

```
service.name        = orders-api
service.namespace   = commerce
service.instance.id = <runtime instance identity>
service.version     = <deployment version>
cloud.resource_id   = <App Service Azure resource ID>
```

Application Insights derives Cloud Role Name from `service.namespace` plus `service.name`, and Cloud Role Instance from `service.instance.id`; these values drive Application Map nodes. Resource detectors can populate Azure context, but explicit stable names prevent one logical service from appearing as multiple components. [6] [7] [8]

---

5. Asynchronous Messaging

Azure Service Bus

Service Bus defines a `Diagnostic-Id` message property using W3C trace-parent format. The current .NET `ServiceBusProcessor` provides instrumentation for send, receive/process, settlement, defer and dead-letter operations; handler-based processing is instrumented. The product documentation still describes the .NET client's direct OpenTelemetry support as **experimental**, so validate the exact SDK/runtime combination rather than assuming parity across languages. [19]

Recommended message metadata:

```
Diagnostic-Id            = injected trace context used by supported SDKs
business-correlation-id  = durable invocation ID
MessageId                = immutable message identity
causation-id             = identifier of command/event that produced this message
```

Use Service Bus `CorrelationId` only if its request/reply or conversation semantics align with your domain; otherwise use an application property to avoid conflating broker reply correlation with business invocation correlation. Service Bus supports both predefined broker properties and arbitrary user properties. [4]

Retries should create distinct processing spans but retain the message/business identifiers. Preserve the same metadata when dead-lettering and when re-submitting; include the original `MessageId` or causation identifier rather than pretending the re-submission is the same transport delivery.

Event Grid

Event Grid client libraries for .NET, Java, Python and JavaScript set CloudEvent `traceparent` and `tracestate` extension attributes when distributed tracing is enabled. For Event Grid delivery to Service Bus or Event Hubs, Microsoft documents mapping the CloudEvent `traceparent` extension to the AMQP `Diagnostic-Id` application property. [20]

For fan-out or batch consumption, use OpenTelemetry **span links** from each consumer/process span to the message creation context. OTel messaging conventions specify links as the default correlation mechanism because one span can have only one parent and batches or multiple consumers can have multiple causal predecessors. [5]

Azure Queue Storage

Queue Storage messages expose body, message ID, dequeue count and timing properties, but no per-message application-property bag comparable to Service Bus. [21]

Therefore use a versioned message envelope:

```
{
  "schemaVersion": 1,
  "traceparent": "00-...",
  "tracestate": "...",
  "businessCorrelationId": "...",
  "messageId": "...",
  "causationId": "...",
  "payload": {}
}
```

The Azure SDK call to enqueue/dequeue can still appear as a dependency through Azure SDK instrumentation, but the worker must explicitly extract the envelope context and start its consumer/process span.

---

6. Data Dependencies

Application Insights can automatically collect supported HTTP, SQL and Azure SDK dependency calls and show them in Application Map and Transaction Diagnostics. Unsupported dependencies can be represented with a custom client span. [22]

Important distinction:

- **Instrumented HTTP service:** client dependency span plus downstream server request span; full distributed participation.
- **SQL/database without server trace participation:** client dependency span only. This proves that the application made the database call and records duration/result, but not an internal datastore span tree.
- **Cosmos DB:** supported .NET and Java SDKs expose OTel-compatible database and Cosmos-specific attributes, including operation, container, status, request charge and contacted regions. [23]

Do not describe a client-side SQL dependency as "the database participated in W3C tracing" unless the datastore actually accepts and emits compatible server trace context.

---

7. Public Preview Options

Azure Monitor Traces and Trace Analytics — Public Preview

Azure Monitor Traces provides a dedicated trace store that assembles spans into complete traces at ingestion. Trace Analytics adds trace-level search, P50/P99 analysis, span statistics, critical-path analysis, attribute comparison and trace-derived metrics. It is explicitly **Preview**, has no SLA, is not recommended as the sole production diagnostic experience, and has regional onboarding limitations. [24]

Use it as an additional analysis surface, not as a prerequisite for the GA architecture. Keep GA Transaction Diagnostics, Application Map and KQL as the operational baseline.

App Service Windows-container autoinstrumentation — Public Preview

If any App Service tier is deployed as a Windows container, autoinstrumentation is currently marked Public Preview. For a production-critical correlation implementation, prefer code-based instrumentation until that hosting combination becomes GA. [9]

---

8. Concrete Header and Attribute Convention

Purpose	HTTP	Service Bus	Event Grid	Queue Storage	Telemetry
Technical trace	`traceparent`	`Diagnostic-Id` /SDK context	CloudEvent `traceparent`	Envelope `traceparent`	`OperationId`
Vendor trace state	`tracestate`	SDK/ application property as supported	CloudEvent `tracestate`	Envelope `tracestate`	Exporter-managed
Durable invocation	`x-business-correlation-id`	`business-correlation-id` application property	`businesscorrelationid` extension	Envelope field	`Properties["business.correlation_id"]`
Message identity	—	`MessageId`	Event `id`	Envelope `messageId`	`messaging.message.id`
Causation	—	`causation-id`	Extension	Envelope field	`messaging.message.conversation_id` or custom attribute

Use baggage only within controlled trust boundaries. OTel baggage is automatically propagated by many instrumentations but is not automatically added to spans/logs, has no built-in integrity protection, and can leak to unintended downstream services. [25]

9. Verification Procedure

1. **Disable sampling for the test path**
   - Browser sampling 100%.
   - APIM diagnostic sampling 100% and always log errors.
   - All service samplers `always_on` or equivalent.
   - Microsoft distros now default to rate-limited sampling—approximately five traces per second—unless overridden. [6] [26]
2. **Invoke with known values**
   - Send a valid controlled `x-business-correlation-id`.
   - Capture the browser's first outbound `traceparent`.
3. **Inspect every hop**
   - The 32-hex `trace-id` must remain constant.
   - Each participating hop must emit a different span `Id`.
   - Each child `ParentId` must match the preceding request/dependency `Id`.
   - The business ID must remain unchanged.
4. **Use Application Insights**
   - Open Transaction Diagnostics from the APIM or backend request.
   - Confirm browser dependency → APIM request → APIM backend dependency → service request → datastore dependency.
   - Use Application Map to verify distinct, stable role names and expected edges. [8] [27]
5. **Exercise asynchronous cases**
   - Normal delivery.
   - Retry/abandon.
   - Dead-letter and controlled reprocessing.
   - Topic/Event Grid fan-out.
   - Delayed processing after the original HTTP request completes.

- Verify technical trace/link behavior and business-ID continuity independently.

KQL: reconstruct one W3C operation

```
let traceId = "<32-hex-trace-id>";
union withsource=TableName
    AppPageViews,
    AppRequests,
    AppDependencies,
    AppTraces,
    AppExceptions
| where OperationId == traceId
| project
    TimeGenerated,
    TableName,
    AppRoleName,
    AppRoleInstance,
    OperationId,
    Id = column_ifexists("Id", ""),
    ParentId,
    Name = column_ifexists("Name", ""),
    Message = column_ifexists("Message", ""),
    Properties
| order by TimeGenerated asc
```

`OperationId`, `Id`, `ParentId`, role identity and `Properties` are current fields in the corresponding workspace-based tables. [11] [12] [28] [10]

KQL: find all traces belonging to one durable invocation

```
let businessId = "<business-correlation-id>";
union withsource=TableName
    AppPageViews,
    AppRequests,
    AppDependencies,
    AppTraces,
    AppExceptions
| extend BusinessCorrelationId =
    tostring(Properties["business.correlation_id"])
| where BusinessCorrelationId == businessId
| project
    TimeGenerated,
    TableName,
    AppRoleName,
    OperationId,
    ParentId,
    BusinessCorrelationId,
```

```
    Name = column_ifexists("Name", ""),
    Message = column_ifexists("Message", "")
| order by TimeGenerated asc
```

KQL: detect missing parents within collected spans

```
 let traceId = "<32-hex-trace-id>";
let spans = materialize(
    union AppRequests, AppDependencies
    | where OperationId == traceId
    | project TimeGenerated, AppRoleName, OperationId, Id, ParentId, Name
);
spans
| where isnotempty(ParentId)
| extend JoinKey = strcat(OperationId, ":", ParentId)
| join kind=leftanti (
    spans
    | extend JoinKey = strcat(OperationId, ":", Id)
    | project JoinKey
) on JoinKey
| project TimeGenerated, AppRoleName, Name, OperationId, ParentId
```

An apparent orphan can indicate the true root, sampling, delayed ingestion, a resource outside the query scope, unsupported instrumentation, or broken propagation—not automatically an application defect.

---

10. Alternatives and Final Decision

Do not introduce a third-party APM for the baseline. Azure-native GA capabilities cover W3C propagation, browser-to-backend correlation, APIM gateway spans, App Service services, standard dependencies, topology and transaction diagnostics.

An **OpenTelemetry Collector/secondary OTLP export** is justified only when you need centralized redaction, routing, tail-sampling, multi-backend export, or portability beyond Azure Monitor. Microsoft's .NET distro can export simultaneously to Azure Monitor and OTLP; Node.js and Python can add OTLP span processors. Microsoft does not support the downstream third-party OTLP components or experiences, so this adds operational and support-boundary complexity. [6]

Final recommendation

Adopt the GA architecture now:

- W3C-only HTTP tracing.
- Microsoft/Azure Monitor OTel distros for services.
- Browser AI JavaScript SDK with constrained CORS correlation.
- APIM diagnostics explicitly set to `W3C`.
- Stable service/resource attributes.
- Azure SDK messaging instrumentation where documented.
- Explicit message-envelope propagation where not automatic.
- A separate edge-generated `x-business-correlation-id`.
- 100% sampling for certification tests, followed by deliberately coordinated production sampling.
- KQL and Transaction Diagnostics as the formal verification evidence.
- Azure Monitor Traces/Trace Analytics only as an additional Preview capability.